

PRODUCT INVENTORY MANAGEMENT SYSTEM

(A MERN Stack Based Web Application for Inventory Management)

A 15 DAYS INTERNSHIP REPORT

Submitted by

PANCHAL VISHAL

(Enrolment Number: 230220131144)

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

Computer Science & Engineering

7Th Semester



**GUJARAT TECHNOLOGICAL UNIVERSITY,
AHMEDABAD**



July, 2026



GOVERNMENT ENGINEERING COLLEGE, PATAN

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the internship report submitted along with the project entitled "PRODUCT INVENTORY MANAGEMENT SYSTEM" has been carried out by Panchal Vishal (Enrolment No: 230220131144) under my guidance in partial fulfillment for the degree of Bachelor of Engineering in Computer Science & Engineering, 7th Semester of Gujarat Technological University, Ahmedabad, during the academic year 2025-26.

Internal Guide

Head of the Department



GOVERNMENT ENGINEERING COLLEGE, PATAN

DECLARATION

I hereby declare that the Internship report submitted along with the Internship entitled "PRODUCT INVENTORY MANAGEMENT SYSTEM" submitted in partial fulfillment for the degree of Bachelor of Engineering in Computer Science & Engineering to Gujarat Technological University, Ahmedabad, is a bonafide record of original project work carried out by me at BrainyBeam Info-Tech Pvt. Ltd. under the supervision of Mr. Sagar Jasani (Industry Guide) and my Internal Guide, and that no part of this report has been directly copied from any student's report or taken from any other source, without providing due reference.

Name of the Student

Sign: _____

Panchal Vishal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Gujarat Technological University for including a mandatory 15-day internship as part of the 7th Semester curriculum, which gave me the opportunity to apply theoretical knowledge to a real, practical project.

I am deeply thankful to my Internal Guide for continuous supervision, valuable suggestions, and encouragement throughout the internship period. Their guidance was instrumental in shaping the direction of this project and in helping me overcome the technical challenges faced during development.

I am also sincerely grateful to Mr. Sagar Jasani, CEO of BrainyBeam Info-Tech Pvt. Ltd., Ahmedabad, and to my industry mentor at the company, for their support, technical guidance, and for providing me the opportunity to work on a real-world MERN stack project during this internship.

I would also like to thank the Head of the Department and all the faculty members of the Computer Science & Engineering Department, Government Engineering College, Patan, for providing the infrastructure, lab facilities, and a supportive environment necessary for the successful completion of this internship.

Finally, I extend my heartfelt thanks to my family and friends for their constant motivation and support during this internship, and to all those who directly or indirectly contributed to the successful completion of this project.

Panchal Vishal

Enrolment No: 230220131144

ABSTRACT

Manual tracking of stock levels, purchases, and supplier records using spreadsheets or paper registers is slow, error-prone, and difficult to scale as the number of products and transactions grows. The objective of this internship was to design and develop a “Product Inventory Management System” — a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) — that digitizes and automates the core inventory workflow of a small-to-medium business: adding and categorizing products, recording suppliers, tracking stock-in and stock-out movements, flagging low-stock items, and generating summary reports.

The primary objective of the internship was backend development. A RESTful API was designed and implemented using Node.js and Express.js, exposing endpoints for authentication, product management, category and supplier management, and stock-movement logging. MongoDB, accessed through the Mongoose ODM, was used as the persistence layer. JSON Web Tokens (JWT) were used to secure the API and implement role-based access control. A pre-built React admin dashboard template was customized and wired to the backend APIs to serve as the frontend, allowing development effort to be concentrated on backend logic, database design, and API security within the 15-day internship period.

This report documents the complete internship work carried out over 15 days at BrainyBeam Info-Tech Pvt. Ltd., Ahmedabad, covering the company and department overview, project planning, system analysis and design, implementation, testing, and the results obtained, along with the advantages, limitations, and future scope of the platform.

Keywords: MERN Stack, MongoDB, Express.js, React.js, Node.js, REST API, JWT Authentication, Inventory Management, Stock Tracking, Role-Based Access Control

TABLE OF CONTENTS

CHAPTER 1: OVERVIEW OF THE COMPANY	1
1.1 HISTORY.....	1
1.2 DIFFERENT PRODUCTS / SCOPE OF WORK.....	1
1.3 ORGANIZATION CHART	1
1.4 CAPACITY / SCALE OF OPERATIONS.....	1
CHAPTER 2: OVERVIEW OF DEPARTMENT AND PROCESS LAYOUT.....	2
2.1 WORK CARRIED OUT IN THE WEB DEVELOPMENT DEPARTMENT.....	2
2.2 TECHNICAL SPECIFICATIONS OF TOOLS / TECHNOLOGIES USED	2
2.3 SCHEMATIC LAYOUT OF THE DEVELOPMENT PROCESS	2
2.4 STAGES OF DEVELOPMENT EXPLAINED.....	3
CHAPTER 3: INTRODUCTION TO INTERNSHIP / PROJECT AND PROJECT MANAGEMENT	4
3.1 PROJECT / INTERNSHIP SUMMARY	4
3.2 PURPOSE	4
3.3 OBJECTIVE	4
3.4 SCOPE	4
3.5 TECHNOLOGY AND LITERATURE REVIEW	4
3.6 PROJECT / INTERNSHIP PLANNING	5
3.6.1 Development Approach and Justification.....	5
3.6.2 Effort, Time and Cost Estimation.....	5
3.6.3 Roles and Responsibilities.....	5
3.6.4 Group Dependencies	5
3.7 PROJECT / INTERNSHIP SCHEDULING	5
CHAPTER 4: SYSTEM ANALYSIS.....	7
4.1 STUDY OF CURRENT SYSTEM	7
4.2 PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM	7
4.3 REQUIREMENTS OF NEW SYSTEM.....	7
4.4 SYSTEM FEASIBILITY.....	7
4.4.1 Organizational Objective Contribution	7
4.4.2 Technology, Cost and Schedule Feasibility	7
4.4.3 Integration with Other Systems.....	7
4.5 ACTIVITY / PROCESS IN NEW SYSTEM.....	8
4.6 FEATURES OF NEW SYSTEM.....	8
4.7 LIST OF MAIN MODULES.....	9
4.8 SELECTION OF HARDWARE / SOFTWARE / TECHNOLOGIES AND JUSTIFICATION	9
CHAPTER 5: SYSTEM DESIGN	10
5.1 SYSTEM DESIGN & METHODOLOGY	10
5.2 DATABASE DESIGN	10

5.3 INPUT / OUTPUT AND INTERFACE DESIGN	12
5.3.1 State Transition Diagram	12
5.3.2 Samples of Forms, Reports and Interface.....	12
5.3.3 Access Control / Mechanism / Security.....	13
CHAPTER 6: IMPLEMENTATION	14
6.1 IMPLEMENTATION PLATFORM / ENVIRONMENT	14
6.2 PROCESS / PROGRAM / TECHNOLOGY / MODULE SPECIFICATIONS	14
6.3 FINDINGS / RESULTS / OUTCOMES.....	14
6.4 RESULT ANALYSIS / COMPARISON / DELIBERATIONS	16
CHAPTER 7: TESTING	17
7.1 TESTING PLAN / STRATEGY.....	17
7.2 TEST RESULTS AND ANALYSIS	17
7.2.1 Test Cases	17
CHAPTER 8: CONCLUSION AND DISCUSSION.....	18
8.1 OVERALL ANALYSIS OF INTERNSHIP VIABILITY	18
8.2 PHOTOGRAPHS AND DATE OF SURPRISE VISIT BY INSTITUTE MENTOR	18
8.3 PROBLEMS ENCOUNTERED AND POSSIBLE SOLUTIONS.....	18
8.4 SUMMARY OF INTERNSHIP WORK	18
8.5 LIMITATIONS AND FUTURE ENHANCEMENT.....	18
REFERENCES	20

CHAPTER 1: OVERVIEW OF THE COMPANY

1.1 HISTORY

BrainyBeam Info-Tech Pvt. Ltd. is a software development and IT services company headquartered at 7th Floor, Premchand House, Near Old High Court Way, Behind ICICI Bank, Ashram Road, Ahmedabad – 380009, Gujarat, India. The company operates under the tagline “Better Brains for your Idea” and is recognized under AICTE and MSME. Founded to provide custom web and mobile application development services, the company has, over the years, built a portfolio of client projects spanning e-commerce, business websites, and custom web applications, and conducts structured internship and training programs for engineering students in modern web technologies.

1.2 DIFFERENT PRODUCTS / SCOPE OF WORK

BrainyBeam Info-Tech Pvt. Ltd. offers a range of services including custom website design and development, e-commerce solutions, mobile application development, UI/UX design, and full-stack web application development using modern JavaScript frameworks such as the MERN stack (MongoDB, Express.js, React.js, Node.js). The company also provides API development, third-party integrations, and website maintenance and support services to clients across various industries.

1.3 ORGANIZATION CHART

The company follows a flat, project-team-based organizational structure, led by the CEO, Mr. Sagar Jasani, with reporting technical leads and project managers overseeing individual development teams, each comprising frontend developers, backend developers, and quality-assurance/testing staff. Interns are attached to a development team and report to an assigned industry mentor within that team.

1.4 CAPACITY / SCALE OF OPERATIONS

BrainyBeam Info-Tech Pvt. Ltd. operates as a mid-sized IT services company from its Ahmedabad office, handling multiple concurrent client projects across web and mobile development, and regularly hosts structured internship batches for engineering students from various GTU-affiliated colleges.

CHAPTER 2: OVERVIEW OF DEPARTMENT AND PROCESS LAYOUT

2.1 WORK CARRIED OUT IN THE WEB DEVELOPMENT DEPARTMENT

The internship was carried out within the company's web development department, which is responsible for designing, developing, testing, and deploying full-stack web applications for clients using the MERN stack. The department's day-to-day work includes requirement discussions, sprint-based development, code reviews, and API testing.

2.2 TECHNICAL SPECIFICATIONS OF TOOLS / TECHNOLOGIES USED

Tool / Technology	Purpose
Visual Studio Code	Primary IDE for backend and frontend development
Node.js	JavaScript runtime for the backend server
Express.js	Web framework for building the REST API
MongoDB Atlas	Cloud-hosted NoSQL database
Mongoose	Object Data Modelling (ODM) library for MongoDB
Postman	API endpoint testing and documentation
Git & GitHub	Version control and source code management

2.3 SCHEMATIC LAYOUT OF THE DEVELOPMENT PROCESS

The department follows an iterative development process: requirement gathering, database schema design, API development, integration with the frontend template, testing, and deployment, as illustrated below.

Fig. 4.2 DFD Level 0 (Context Diagram)

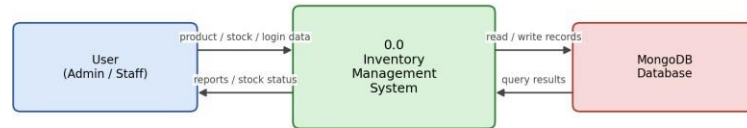


Fig. 2.1 Schematic Layout of the MERN Stack Development Process

2.4 STAGES OF DEVELOPMENT EXPLAINED

Requirement Gathering: understanding the client/internship problem statement and desired features. Schema Design: modelling MongoDB collections and their relationships using Mongoose. API Development: building and testing Express.js routes and controllers for each resource. Frontend Integration: wiring a React admin template to the backend via Axios. Testing: verifying functional correctness and security using Postman and manual testing. Deployment: hosting the API and frontend on cloud platforms.

CHAPTER 3: INTRODUCTION TO INTERNSHIP / PROJECT AND PROJECT MANAGEMENT

3.1 PROJECT / INTERNSHIP SUMMARY

This internship involved designing and developing a Product Inventory Management System using the MERN stack, with development effort concentrated on the backend (Node.js, Express.js, MongoDB) while a pre-built admin dashboard template was customized for the frontend. The system allows authorized users to manage products, categories, and suppliers, track stock movements, and view low-stock alerts and summary reports through a secure, role-based REST API.

3.2 PURPOSE

The purpose of the project is to replace manual, spreadsheet-based inventory tracking with a centralized, secure, web-based system that keeps stock quantities accurate in real time and provides visibility into low-stock products and supplier relationships.

3.3 OBJECTIVE

- To design a MongoDB schema modelling products, categories, suppliers, and stock transactions.
- To build a secure REST API using Node.js and Express.js for all inventory operations.
- To implement JWT-based authentication and role-based access control.
- To integrate a customized React admin dashboard template as the frontend.
- To provide low-stock alerts and summary inventory reports.

3.4 SCOPE

The scope of the 15-day internship covers the backend REST API, MongoDB data model, JWT authentication and role-based access control, and integration with a React admin template frontend. It does not cover barcode scanning, automated purchase ordering, or multi-warehouse support, which are identified as future scope in Chapter 8.

3.5 TECHNOLOGY AND LITERATURE REVIEW

A brief survey of existing inventory tracking approaches — spreadsheet-based tracking, commercial ERP/inventory modules such as Zoho Inventory and Odoo Inventory, and open-source MERN-stack reference projects — was carried out to inform the design of this system's data model and REST API structure. Spreadsheets lack access control and real-time updates; commercial ERP suites are often heavier and less customizable than needed for a small team; open-source MERN reference projects informed the folder structure and schema-modelling patterns adopted here.

3.6 PROJECT / INTERNSHIP PLANNING

3.6.1 Development Approach and Justification

An iterative, module-wise development approach was adopted, building and testing one module (authentication, then products, then categories/suppliers, then stock tracking, then reporting) at a time, which allowed early modules to be verified before later modules that depended on them were built.

3.6.2 Effort, Time and Cost Estimation

The entire internship was scoped to fit within 15 working days, with backend development (schema design, API, authentication, security) allotted roughly 8 days, frontend template integration 3 days, and testing/documentation the remaining 4 days. As an academic internship, no direct cost was incurred beyond the student's and mentor's time.

3.6.3 Roles and Responsibilities

As an individual internship, the intern (Panchal Vishal) was responsible for all backend development, database schema design, and frontend template integration, under the technical guidance and periodic review of the industry mentor at BrainyBeam Info-Tech Pvt. Ltd. and the Internal Guide at the college.

3.6.4 Group Dependencies

This was an individually carried out internship project; there were no group dependencies. Coordination was limited to periodic check-ins with the industry mentor and internal faculty guide.

3.7 PROJECT / INTERNSHIP SCHEDULING

Day(s)	Activity
Day 1-2	Requirement analysis, company/department orientation, and system design
Day 3-5	Project scaffolding, MongoDB schema design, and authentication module
Day 6-9	Product, category, supplier, and stock-tracking REST APIs
Day 10-12	Role-based access control, reporting endpoints, and frontend template integration
Day 13-15	Integration testing, bug-fixing, and report preparation

CHAPTER 4: SYSTEM ANALYSIS

4.1 STUDY OF CURRENT SYSTEM

Prior to this project, inventory at a small business scale is typically tracked using spreadsheets or paper ledgers, with product, stock, and supplier information recorded manually by staff.

4.2 PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM

- No access control — anyone with the file can alter stock records.
- No real-time visibility into current stock levels across users.
- No automated alerting when stock runs low.
- Difficult to trace which supplier a batch of stock came from.
- Error-prone manual calculations of stock quantities.

4.3 REQUIREMENTS OF NEW SYSTEM

- Centralized, authenticated access to product, category, supplier, and stock data.
- Automatic stock-quantity updates on every recorded transaction.
- Role-based restrictions on sensitive operations.
- Low-stock alerting and summary reporting.

4.4 SYSTEM FEASIBILITY

4.4.1 Organizational Objective Contribution

The system directly supports the objective of reducing manual effort and stock discrepancies in inventory tracking, a common operational need for small and medium businesses.

4.4.2 Technology, Cost and Schedule Feasibility

The MERN stack is free and open-source, well-documented, and widely supported, making the system technically and economically feasible to build within the 15-day internship schedule.

4.4.3 Integration with Other Systems

Because the system exposes a REST API, it can be integrated with other systems (e.g. a future point-of-sale or e-commerce platform) without requiring changes to the underlying data model.

Feasibility Aspect	Assessment
Technical	Feasible — standard MERN stack, well-documented tooling
Economic	Feasible — open-source stack, no licensing cost
Schedule	Feasible within 15-day internship period
Operational	Feasible — REST API allows future integration

4.5 ACTIVITY / PROCESS IN NEW SYSTEM

The Level 1 activity diagram below decomposes the system into its five core processes: Authentication, Product Management, Category & Supplier Management, Stock/Inventory Tracking, and Reporting & Dashboard, each interacting with the shared MongoDB database.

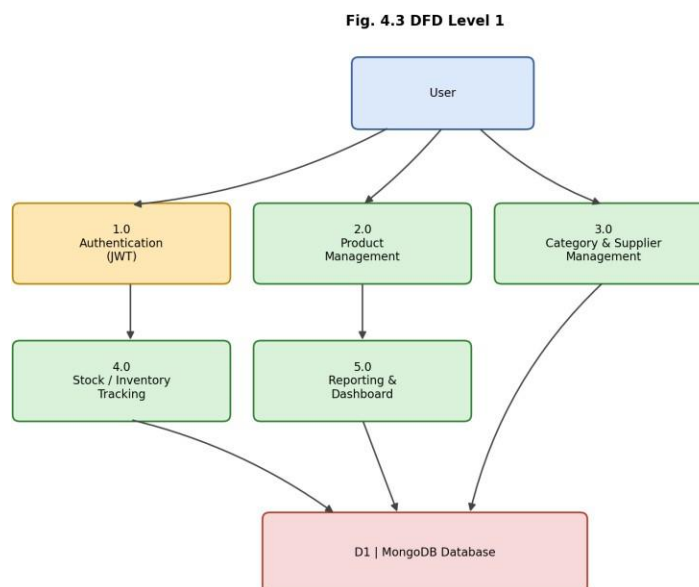


Fig. 4.2 Activity / Process Diagram — DFD Level 1

4.6 FEATURES OF NEW SYSTEM

- Secure JWT-based login and role-based access control.
- Full CRUD for products, categories, and suppliers.
- Stock-in/stock-out transaction logging with automatic quantity updates.

- Low-stock alerting based on a configurable reorder threshold.
- Summary reporting endpoints consumed by a dashboard UI.

4.7 LIST OF MAIN MODULES

- Authentication Module
- Product Management Module
- Category & Supplier Management Module
- Stock / Inventory Tracking Module
- Role-Based Access Control Module
- Reporting & Dashboard Module

4.8 SELECTION OF HARDWARE / SOFTWARE / TECHNOLOGIES AND JUSTIFICATION

Node.js and Express.js were selected for the backend due to their non-blocking I/O model (suited to a request-heavy REST API), large ecosystem, and use of JavaScript across the stack. MongoDB was chosen over a relational database because the product/category/supplier/stock-log data model benefits from MongoDB's flexible document structure and straightforward reference-based relationships via Mongoose. JWT was selected for authentication because it is stateless and works well with a REST API consumed by a single-page React frontend.

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM DESIGN & METHODOLOGY

The system follows a layered client-server architecture. The client layer, a React.js single-page application built on a customized admin dashboard template, communicates with the server over HTTP using Axios. The server layer, built with Node.js and Express.js, authenticates each request through JWT middleware and enforces role-based access control, before routing the request to the appropriate controller, which interacts with MongoDB through Mongoose models.

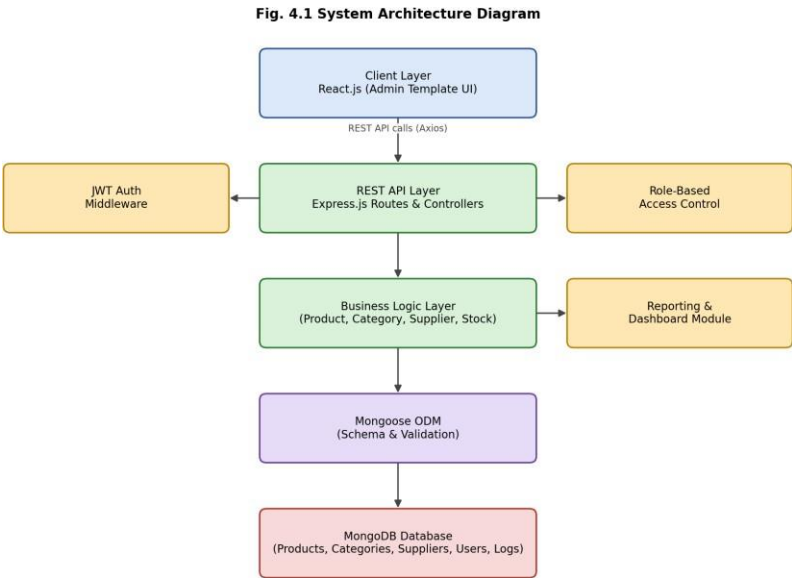


Fig. 5.1 System Architecture Diagram

5.2 DATABASE DESIGN

The MongoDB schema, designed via Mongoose, defines the User, Product, Category, Supplier, and StockLog collections along with their relationships, as shown in the Entity Relationship diagram and summarized in the table below.

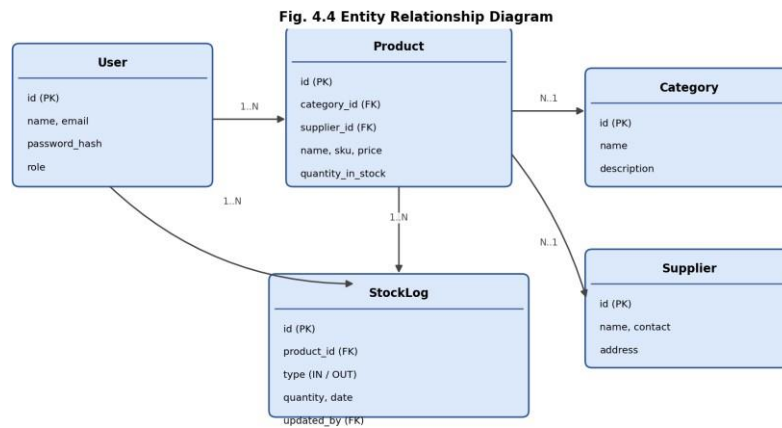


Fig. 5.2 Entity Relationship (ER) Diagram

Table 5.1 Database Schema — Collections and Key Fields

Collection	Key Fields	Relation
User	_id, name, email, password_hash, role	1 — N StockLog
Category	_id, name, description	1 — N Product
Supplier	_id, name, contact, address	1 — N Product
Product	_id, name, sku, price, category_id (ref), supplier_id (ref), quantity_in_stock	1 — N StockLog
StockLog	_id, product_id (ref), type (IN/OUT), quantity, date, updated_by (ref)	N — 1 Product, N — 1 User

The primary actors interacting with the system are Admin and Staff/Manager, with use cases including Login/Authenticate, Manage Products, Manage Categories, Manage Suppliers, Update Stock, View Low-Stock Alerts, Generate Reports, and Manage Users & Roles (Admin only).

Fig. 4.5 Use Case Diagram

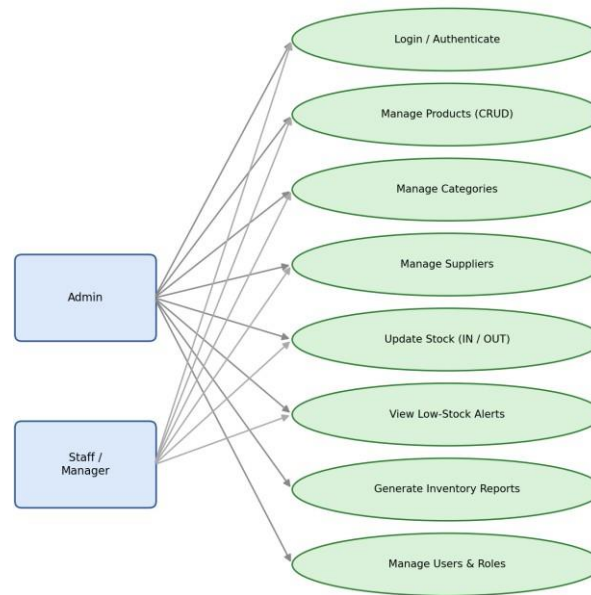


Fig. 5.3 Use Case Diagram

5.3 INPUT / OUTPUT AND INTERFACE DESIGN

5.3.1 State Transition Diagram

Each product moves between three stock states — In Stock, Low Stock, and Out of Stock — based on its current quantity relative to a configured reorder threshold, transitioning on every stock-in or stock-out event.

Fig. 5.1 State Transition Diagram — Product Stock Status

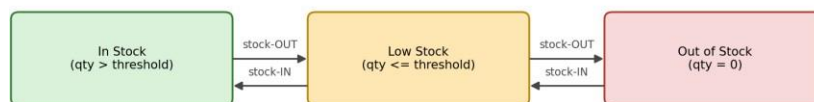


Fig. 5.4 State Transition Diagram — Product Stock Status

5.3.2 Samples of Forms, Reports and Interface

The React dashboard (built on the customized admin template) presents a login form, a paginated product table with add/edit/delete actions, category and supplier management

forms, a stock-update form (product, quantity, IN/OUT), and a reporting dashboard summarizing total stock value, low-stock products, and recent stock transactions.

5.3.3 Access Control / Mechanism / Security

Every protected API route is guarded by JWT authentication middleware, which verifies the bearer token and attaches the authenticated user's id and role to the request. A role-guard middleware then restricts sensitive operations (e.g. deleting a product, managing users) to Admin accounts only. Passwords are hashed using bcrypt before storage.

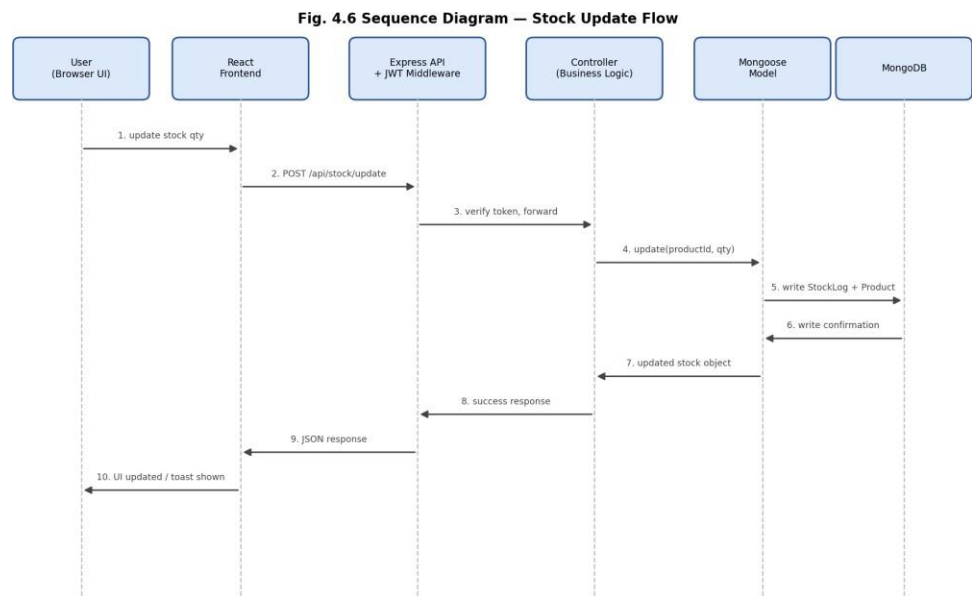


Fig. 5.5 Sequence Diagram — Stock Update Flow

CHAPTER 6: IMPLEMENTATION

6.1 IMPLEMENTATION PLATFORM / ENVIRONMENT

Category	Technology / Tool
Operating System	Windows 10/11
Backend Runtime	Node.js
Backend Framework	Express.js
Database	MongoDB Atlas (via Mongoose ODM)
Frontend	React.js (customized admin dashboard template) + Axios
Authentication	JSON Web Tokens (JWT), bcrypt password hashing
API Testing	Postman
Version Control	Git & GitHub
IDE	Visual Studio Code
Deployment	Render (API), MongoDB Atlas (database), Vercel (frontend)

6.2 PROCESS / PROGRAM / TECHNOLOGY / MODULE SPECIFICATIONS

The backend follows a feature-based, MVC-style folder structure: /models (Mongoose schemas), /routes (Express route definitions), /controllers (business logic per resource), /middleware (JWT authentication and role-guard middleware), and /config (database connection and environment configuration). Each REST endpoint validates its request body, performs the relevant Mongoose query, and returns a consistent JSON response with an appropriate HTTP status code. Stock updates atomically create a StockLog entry and adjust the referenced Product's quantity_in_stock field within the same operation.

6.3 FINDINGS / RESULTS / OUTCOMES

The completed system allows an authenticated user to add products, categorize and assign suppliers to them, record stock movements, and immediately see updated stock levels and low-stock alerts on the dashboard. All planned modules — authentication, product/category/supplier management, stock tracking, role-based access control, and reporting — were implemented and verified working within the 15-day internship.

The screens below are illustrative mockups of the dashboard UI, reconstructed to represent the implemented modules; the student should replace these with actual screenshots captured from the running application before final submission.

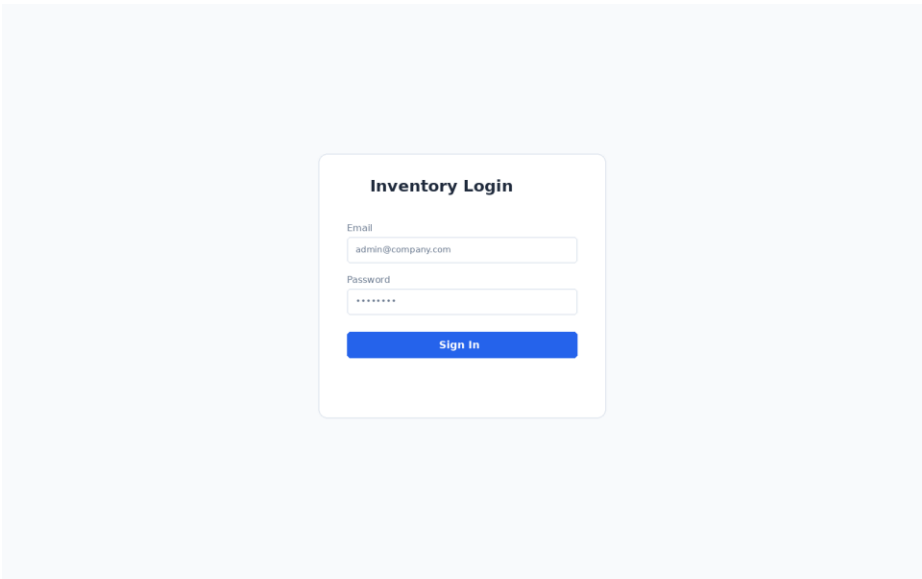


Fig. 6.1 Login Screen



Fig. 6.2 Dashboard Overview

Inventory Admin	Product Management						Admin User
							+ Add Product
	SKU	Name	Category	Stock	Price	Status	
	SKU-1001	Mechanical Keyboard	Electronics	48	₹2,499	In Stock	
	SKU-1002	Wireless Mouse	Electronics	2	₹899	Low Stock	
	SKU-1003	Office Chair	Furniture	15	₹5,999	In Stock	
	SKU-1004	USB-C Cable	Accessories	4	₹299	Low Stock	
	SKU-1005	Desk Lamp	Furniture	30	₹1,199	In Stock	
	SKU-1006	HDMI Adapter	Accessories	5	₹499	Low Stock	
	SKU-1007	Monitor Stand	Furniture	22	₹1,799	In Stock	

Fig. 6.3 Product Listing Screen

Inventory Admin	Add New Product	Admin User
Dashboard Products Categories Suppliers Stock Log Reports Users Logout	Product Name Mechanical Keyboard SKU SKU-1008 Category Electronics Supplier TechSource Pvt. Ltd. Quantity in Stock 60 Reorder Threshold 10 Unit Price (₹) 2499 Save Cancel	

Fig. 6.4 Add / Edit Product Form

6.4 RESULT ANALYSIS / COMPARISON / DELIBERATIONS

Compared to the spreadsheet-based tracking studied in Chapter 4, the implemented system provides real-time, access-controlled stock updates and automated low-stock alerts, meeting the core requirements identified at the start of the internship. The reliance on a pre-built frontend template allowed the available time to be concentrated on backend correctness and security, at the cost of some UI customization depth, which is identified as an area for future polish in Chapter 8.

CHAPTER 7: TESTING

7.1 TESTING PLAN / STRATEGY

Testing covered unit-level validation of controller logic, integration testing of the full authenticated request path, API testing of every endpoint using Postman, and security testing to confirm that role-based restrictions and JWT validation were correctly enforced.

7.2 TEST RESULTS AND ANALYSIS

7.2.1 Test Cases

Table 7.1 Test Cases and Results

Test ID	Test Condition	Expected Output	Status
T01	Register a new user	User created, password hashed	Pass
T02	Login with valid credentials	JWT token issued	Pass
T03	Login with invalid credentials	401 Unauthorized returned	Pass
T04	Access protected route without token	401 Unauthorized returned	Pass
T05	Staff user attempts to delete a product	403 Forbidden returned	Pass
T06	Add a product with valid data	Product saved, appears in listing	Pass
T07	Add a product with missing required field	Validation error returned	Pass
T08	Record a stock-IN transaction	StockLog created, quantity increased	Pass
T09	Stock-OUT exceeding available stock	Validation error returned	Pass
T10	Fetch low-stock report	Only products below threshold returned	Pass

CHAPTER 8: CONCLUSION AND DISCUSSION

8.1 OVERALL ANALYSIS OF INTERNSHIP VIABILITY

The internship successfully delivered a working Product Inventory Management System within the planned 15-day schedule, meeting all functional objectives set out in Chapter 3. The project demonstrated that a lightweight, secure inventory system can be built end-to-end using the MERN stack by a single developer within a short, structured internship period.

8.2 PHOTOGRAPHS AND DATE OF SURPRISE VISIT BY INSTITUTE MENTOR

[To be completed by the student: insert photograph(s) taken during the Institute Mentor's surprise visit to BrainyBeam Info-Tech Pvt. Ltd., along with the date of the visit, as provided by the Internal Guide.]

8.3 PROBLEMS ENCOUNTERED AND POSSIBLE SOLUTIONS

- Modelling stock movements without duplicating product data — resolved by introducing a dedicated StockLog collection referencing the Product it affects.
- Keeping a product's displayed stock in sync with its transaction history — resolved by updating the Product's quantity_in_stock atomically whenever a StockLog entry is created.
- Adapting a pre-built admin template's mock data structure to the actual MongoDB schema — resolved by replacing static template data with live Axios calls to the corresponding backend endpoints.

8.4 SUMMARY OF INTERNSHIP WORK

Over the 15-day internship at BrainyBeam Info-Tech Pvt. Ltd., a complete REST API was implemented using Node.js, Express.js, and MongoDB (via Mongoose), with JWT-based authentication and role-based access control, and integrated with a customized React admin dashboard template. The resulting Product Inventory Management System centralizes product, category, supplier, and stock-transaction data, automatically keeping stock quantities up to date and surfacing low-stock alerts and summary reports.

8.5 LIMITATIONS AND FUTURE ENHANCEMENT

- Currently supports a single organization/store; no multi-tenant or multi-warehouse support.
- Stock entry is manual; no barcode or QR-code scanning integration yet.
- No automated purchase-order generation when stock falls below the reorder threshold.
- Future scope includes barcode/QR-code scanning, automated purchase orders, multi-warehouse support, analytics dashboards using MongoDB aggregation pipelines, and role-based audit logging.

REFERENCES

Clerk Documentation 'Authentication for modern applications', available at:
<https://clerk.com/docs>.

Express.js Documentation 'Fast, unopinionated, minimalist web framework for Node.js',
available at: <https://expressjs.com> .

MongoDB, Inc. 'MongoDB Manual', available at: <https://www.mongodb.com/docs> .

Mongoose.js Documentation 'Mongoose ODM v8', available at: <https://mongoosejs.com/docs> .

Node.js Foundation 'Node.js Documentation', available at: <https://nodejs.org/en/docs> .

Postman, Inc. 'Postman Learning Center', available at: <https://learning.postman.com> .

React 'React Documentation', available at: <https://react.dev> .